

This is brilliant! Look at your output: 🟢 Initial Fleet: D0:D8... AA:BB:CC:DD:EE:FF. The ESP32 successfully saved your new MAC address directly to its physical silicon. Even if you unplug it for 10 years, that battery is now permanently authorized.

Now, to answer your excellent question regarding the 60-second timeout:

No, using the second core is absolutely NOT the only option. The problem isn't that we need more processing power; the problem is that our current timer is "dumb." It strictly counts 60 seconds from the moment the drone turns on, regardless of what the human is doing.

We can easily make this a **"Smart Timer"** without using the second core. Here is how professional devices handle this:

The "Connection-Aware" Smart Timer

We will add a tiny bit of logic to the code that watches to see if a phone is actually connected.

1. **If NO phone connects within 60 seconds:** It shuts down the server and goes to Flight Mode.
2. **If a phone DOES connect:** The timer **pauses indefinitely**. You can take 5 minutes to add 20 different batteries. The ESP32 will patiently wait.
3. **The Disconnect Trigger:** The moment you hit "Disconnect" on your phone, the ESP32 instantly says, *"Okay, the human is done,"* shuts down the Admin Server, and jumps straight into Flight Scanner Mode.

This preserves 100% of your processing power for Flight Mode, but gives your technician unlimited time to type!

Here is the updated code to make the timer "Smart".

Step 1: The Smart Timer Code Update

Copy and paste this into your Arduino IDE and upload it.

C++

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <Preferences.h>
```

```
Preferences preferences;
String authorized_macs = "";
```

```
// Smart Timer Variables
```

```

unsigned long bootTime = 0;
const unsigned long ADMIN_TIMEOUT = 60000; // 60 seconds
bool isAdminMode = true;
bool deviceConnected = false; // <--- NEW: Tracks if a phone is connected!

#define SERVICE_UUID      "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHARACTERISTIC_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"

// -----
// 📶 BLUETOOTH CONNECTION TRACKER (NEW)
// -----
class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
        deviceConnected = true;
        Serial.println("📱 Phone Connected! (Timeout timer PAUSED indefinitely)");
    }
    void onDisconnect(BLEServer* pServer) {
        deviceConnected = false;
        Serial.println("📱 Phone Disconnected! Triggering immediate Flight Mode...");
        // Force the timer to expire immediately so it goes to Flight Mode
        bootTime = millis() - ADMIN_TIMEOUT;
    }
};

// -----
// 📶 BLUETOOTH RECEIVER LOGIC
// -----
class MyCallbacks: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
        String rxValue = pCharacteristic->getValue().c_str();
        if (rxValue.length() > 0) {
            rxValue.trim();
            rxValue.toUpperCase();

            if (rxValue.startsWith("+")) {
                String mac = rxValue.substring(1);
                if (authorized_macs.indexOf(mac) == -1) {
                    authorized_macs += mac + ",";
                    preferences.putString("mac_list", authorized_macs);
                    Serial.println("✅ ADDED: " + mac);
                } else {
                    Serial.println("⚠️ MAC already in fleet!");
                }
            }
        }
    }
};

```

```

        else if (rxValue.startsWith("-")) {
            String mac = rxValue.substring(1) + ",";
            if (authorized_macs.indexOf(mac) != -1) {
                authorized_macs.replace(mac, "");
                preferences.putString("mac_list", authorized_macs);
                Serial.println("🗑️ REMOVED: " + mac);
            } else {
                Serial.println("⚠️ MAC not found.");
            }
        } else {
            Serial.println("❌ Invalid Command. Use +MAC or -MAC");
        }
        Serial.println("📶 Current Fleet: " + authorized_macs);
    }
}

// -----
// ⚙️ MAIN SETUP
// -----
void setup() {
    Serial.begin(115200);
    delay(2000);

    preferences.begin("marut_app", false);
    authorized_macs = preferences.getString("mac_list", "D0:D8:9C:8A:89:B4,DE:44:73:F5:43:94,");
    preferences.putString("mac_list", authorized_macs);

    Serial.println("\n" + String("=").substring(0, 40));
    Serial.println("🚀 ESP32-S3 Admin Mode Starting...");
    Serial.println("📶 Initial Fleet: " + authorized_macs);
    Serial.println(String("=").substring(0, 40));

    BLEDevice::init("Marut_Config");
    BLEServer *pServer = BLEDevice::createServer();

    // Attach our new connection tracker!
    pServer->setCallbacks(new MyServerCallbacks());

    BLEService *pService = pServer->createService(SERVICE_UUID);
    BLECharacteristic *pCharacteristic = pService->createCharacteristic(
        CHARACTERISTIC_UUID,
        BLECharacteristic::PROPERTY_READ |

```

```

        BLECharacteristic::PROPERTY_WRITE
    );

    pCharacteristic->setCallbacks(new MyCallbacks());
    pService->start();

    BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
    pAdvertising->addServiceUUID(SERVICE_UUID);
    pAdvertising->setScanResponse(true);
    BLEDevice::startAdvertising();

    bootTime = millis();
    Serial.println("🌐 BLE Server Live! Connect via nRF Connect app now.");
    Serial.println("⌚ 60-second timer started...");
}

// -----
// 🔄 MAIN LOOP
// -----
void loop() {
    // Check if 60 seconds are up, BUT ONLY if no phone is connected!
    if (isAdminMode && !deviceConnected && (millis() - bootTime > ADMIN_TIMEOUT)) {
        Serial.println("\n🔴 Shutting down Admin Server...");

        BLEDevice::deinit(true);
        isAdminMode = false;

        Serial.println("🔒 Switched to Flight Scanner Mode.");
        Serial.println(" (Next step: We will build the scanning logic here!)");
    }

    if (!isAdminMode) {
        delay(1000);
    }
}

```

Test the Magic UX:

1. Upload this code and open the Serial Monitor.
2. Connect your phone via nRF Connect within the first 60 seconds. You will see: 📱 Phone Connected! (Timeout timer PAUSED indefinitely).
3. Add as many MAC IDs as you want. Take 5 minutes if you need to! It won't kick you out.
4. When you are done, **tap the "Disconnect" button** inside the nRF Connect app.

5. Watch the Serial Monitor: The drone instantly detects you disconnected, kills the server, and jumps into Flight Mode immediately!

Let me know if this UX feels production-ready, and then we will write the final Flight Scanner phase!